

AI Development Skills Reference

Architect Skill

name: architect

description: Think through what you are about to build like a senior engineer before writing any code. Surfaces decisions, aligns on language, and produces a clear implementation plan you confirm before anything starts.

You are a senior engineer sitting with a developer before they start building. Your job is not to interrogate them — it is to think alongside them. To ask the questions a senior engineer would ask before letting someone start coding. To catch the things that seem obvious but aren't. To make sure both of you are building the same thing in your heads before either of you touches the code.

This is a thinking session. Not a grilling session.

Step 1 — Understand What's Here

Before saying anything, take stock of what already exists:

- Read the feature description the developer gave you
- Read any context files, documentation, or existing code available
- Build a clear picture of what needs to be built and what already exists

Do not ask about anything already clearly answered by existing documentation. A good senior engineer does their homework before the meeting.

Step 2 — Align on Language

Every project has its own vocabulary. Before discussing implementation, make sure you and the developer mean the same thing by the same words.

Identify 3-5 terms from the feature description that could be interpreted more than one way. Define each one based on what you understand from the context. Present them to the developer for confirmation.

...

Before we think this through — let me make sure we are speaking the same language:

- "[Term]" — I understand this to mean [definition].
Is that right?
- "[Term]" — I am treating this as [definition].
Does that match what you have in mind?

Correct anything that is off before we go further.

...

Update your understanding immediately if the developer corrects a term. Do not continue until the language is aligned.

Step 3 — Think Through the Decisions Together

Now surface the decisions that would meaningfully change what gets built. Not every possible question — only the ones where the answer changes the implementation direction.

A senior engineer knows the difference between a decision that matters and a detail that can be figured out during coding. Ask only what matters.

For each decision:

- Ask one question at a time
- Share what you would do and why — give the developer something to react to, not a blank page to fill
- Listen to their answer before moving to the next decision
- If their answer makes another decision irrelevant — skip it

...

[The decision that needs to be made]

My thinking: [what you would do and the reason behind it]

What do you think — does that approach work for you,
or do you see it differently?

...

Work through decisions in order of impact. The decision that affects the most downstream work comes first.

Step 4 — Know When You Are Done

Stop when every decision that would change the implementation has been resolved. Not when every possible question is answered. When what matters is settled.

A good senior engineer knows when the plan is solid enough to start. They do not keep asking questions for the sake of being thorough.

When you are done, say:

...

Blueprint ready.

...

Step 5 — Produce the Implementation Plan

After saying "Blueprint ready", write a clear implementation plan based on everything discussed.

```

## Implementation Plan — [Feature Name]

### What we are building

[One clear paragraph describing exactly what will be built]

### Language we agreed on

- [Term]: [agreed definition]

- [Term]: [agreed definition]

### Decisions made

- [Decision]: [what was decided and the reasoning]

- [Decision]: [what was decided and the reasoning]

### Assumptions

- [Anything you assumed that was not explicitly confirmed]

### How to build it

[A concise ordered list of implementation steps]

```

Present the plan to the developer. Wait for them to confirm before anything gets built.

Only after explicit confirmation does implementation begin.

What This Session Is Not

This is not an interrogation. You are not trying to catch the developer out or prove their plan is wrong. You are helping them think more clearly before they build.

This is not a specification session. You are not writing a full spec document. You are aligning on the decisions that matter so the implementation can start with confidence.

This is not open-ended. You are not asking questions forever. You are asking what matters, confirming the plan, and getting out of the way so building can begin.

Imprint Skill

name: imprint

description: After building any UI component, extract the visual patterns that matter for consistency and save them to ui-registry.md. So every component built after this one matches what came before.

UI consistency does not happen by accident. It happens because every component is built with awareness of what already exists.

The problem with AI-built interfaces is that each component gets built in isolation. The agent does not remember what it built three sessions ago. So spacing drifts. Colors vary slightly. Border radius is inconsistent. The app looks like it was built by multiple people with different tastes.

This skill fixes that. Run it after building any UI component. It reads what was just built, extracts the patterns that matter for consistency, and saves them so every future component can match.

One command. Run it every time. That is the whole system.

How to Invoke

After building any UI component, run:

```
...  
/imprint  
...
```

To target a specific file:

```
...  
/imprint [filepath]  
...
```

To audit an existing codebase for inconsistencies:

```
...  
/imprint audit  
...
```

If no filepath is given, the skill identifies recently created or modified component files automatically and captures from those.

****When to use audit mode:****

- The project UI already exists and consistency is uncertain
- Multiple sessions have passed without running `/imprint`
- Something looks visually off but it is hard to pinpoint why
- Before establishing `ui-registry.md` for the first time on an existing project

Run `/imprint audit` before running `/imprint` on any project where the UI was not tracked from the beginning.

Step 1 — Find What Was Just Built

If a filepath was provided — read that file directly.

If no filepath was provided — identify which component files were most recently created or modified in this session. Look in the components directory and any other locations where UI files typically live. Read those files.

If it is unclear which files to capture from, ask the developer:

...

Which component should I capture patterns from?

...

Step 2 — Extract What Matters for Consistency

Read the component code. Extract only the classes and values that affect visual consistency across the interface. Not everything — only what makes components look like they belong together.

****Extract these:****

- Background — what bg- class is used for the container, cards, panels
- Border — border color, border width, border style
- Border radius — rounded- class used for this component type
- Text colors — primary text, secondary text, muted text classes
- Text sizes and weights — for headings, body, labels, captions
- Spacing — padding inside the component, gap between elements
- Interactive states — hover, focus, active classes
- Shadow — if used
- Any accent or brand color usage

****Do not extract these:****

- Width and height — too context-dependent to be a consistency rule
- Flex and grid layout — structural, not visual
- Positioning — absolute, relative, z-index — context-dependent
- Animation and transition timing — unless it is a pattern worth enforcing
- Responsive breakpoint variants — capture the base pattern only

Step 3 — Write to ui-registry.md

Open `ui-registry.md`. If it does not exist, create it.

Add a new entry for the component that was captured. Do not overwrite existing entries — append to the registry.

If an entry for this component type already exists — update it rather than duplicating.

Entry format

```markdown

### [Component Name]

File: [filepath]

Last updated: [date]

| Property         | Class           |
|------------------|-----------------|
| -----            | -----           |
| Background       | [class]         |
| Border           | [class]         |
| Border radius    | [class]         |
| Text — primary   | [class]         |
| Text — secondary | [class]         |
| Spacing          | [class]         |
| Hover state      | [class]         |
| Shadow           | [class or none] |
| Accent usage     | [class or none] |

**\*\*Pattern notes:\*\***

[Any important pattern decisions worth noting —  
why a specific class was chosen, what this component  
should always match, what variations are allowed]  
...

#### ### Step 4 — Confirm What Was Captured

After writing to ui-registry.md, confirm to the developer:

...

Imprinted [Component Name] → ui-registry.md

Captured:

- Background: [class]
- Border: [class]
- Radius: [class]
- Text: [classes]
- Spacing: [classes]
- Hover: [class]

Any future component of this type should match these patterns.

...

If anything looked inconsistent or surprising during extraction — flag it:

...

Note: [Something that looked inconsistent or worth the  
developer knowing about]

...

### ### How ui-registry.md Gets Used

The registry is not just a record. It is the consistency enforcer for every future session.

At the start of any session that involves UI work, Claude reads ui-registry.md before writing any component. When building a new card, it checks how existing cards were built. When building a new button, it checks what button patterns already exist. When building a new status badge, it matches the exact classes already in use.

The registry grows as the project grows. The more components are imprinted, the more consistent every new component becomes — because Claude always has a precise reference for what already exists.

### ### The Rule

Build a component. Run `/imprint`. Move on.

Every time. Without exception.

A registry with ten entries is useful. A registry with thirty entries is powerful. A registry that is sometimes updated is unreliable.

Consistency is a habit, not a feature.

### ### Audit Mode — `/imprint audit`

Run this when the UI already exists and consistency is uncertain. Instead of capturing from one component, it scans the entire codebase, finds conflicts, and establishes a clean baseline before any further capturing happens.

#### ##### Step 1 — Scan all UI components

Find every component file in the project. Read each one. Build a complete picture of what visual patterns are currently in use across the entire interface.

#### ##### Step 2 — Identify conflicts

For each visual property that matters for consistency, list every variation found:

...

## UI Consistency Audit

### Conflicts found

**\*\*Border radius\*\***

[List every rounded- variant found and which components use it]

Recommendation: [which one to standardise on and why]

### **\*\*Background colors\*\***

[List every bg- class found — flag any hardcoded hex values]

Recommendation: [which token classes should replace them]

### **\*\*Text colors\*\***

[List every text- color class found — flag any that bypass the design system]

Recommendation: [which token classes should replace them]

### **\*\*Spacing\*\***

[List padding and gap variations found]

Recommendation: [which values to standardise on]

### **\*\*Border colors\*\***

[List every border color class found]

Recommendation: [which token class to standardise on]

### **\*\*Interactive states\*\***

[List hover, focus, active variations found]

Recommendation: [which pattern to standardise on]

### **### Hardcoded values found**

[List every hardcoded hex value, raw color class, or non-token value found — with the file and line where it appears]

These must be replaced with design system tokens.

### **### Recommended baseline**

[The correct pattern for each property — based on what the majority already uses correctly and what the design system defines]

...

### **#### Step 3 — Wait for developer confirmation**

Present the audit report. Do not fix anything. Do not update ui-registry.md yet.

Ask the developer:

...

Audit complete. [X] conflicts found across [Y] properties.

Before I establish the baseline in ui-registry.md:

1. Do the recommendations above look correct?
2. Are there any conflicts you want to resolve differently?
3. Should I flag the hardcoded values as issues to fix?

Confirm the baseline and I will write it to ui-registry.md.

...



#### #### Step 4 — Write the confirmed baseline

After the developer confirms — write the agreed baseline to `ui-registry.md` as the foundation. Label it clearly:

```
```markdown
```

```
## Baseline — Established [date]
```

[Note: This baseline was established via /imprint audit]

```
| Property      | Correct class |
| -----      | -----      |
| Card background | [class]      |
| Card border    | [class]      |
| Card radius    | [class]      |
| Button primary | [class]      |
| Button secondary | [class]     |
| Text primary   | [class]      |
| Text secondary | [class]      |
| Text muted     | [class]      |
| Input background | [class]     |
| Input border   | [class]      |
```
```

#### #### Step 5 — List what needs fixing

After writing the baseline, produce a fix list — every component that deviates from it:

```
```
```

```
## Components to fix
```

These components deviate from the confirmed baseline
and should be updated:

```
- [Component file] — [what is wrong] → [what it should be]
- [Component file] — [what is wrong] → [what it should be]
```
```

The developer can now fix these systematically — or fix them as they encounter each component. Either way, the baseline is established and `/imprint` can be used going forward to keep new components consistent.

```

```

```
Recover Skill
```

```

```

```
name: recover
```

description: When something goes wrong during a build, diagnose what type of failure it is before deciding how to respond. Targeted fix, hard reset, or full rethink — the right response depends on the right diagnosis.

---

Not every problem is a bug. Not every bug needs debugging.

When something goes wrong with AI-assisted development, the instinct is to keep prompting — describe the problem, ask for a fix, get another broken version, describe that problem, ask for another fix. The session gets longer. The context gets polluted. The code gets worse.

The problem is not the code. The problem is not knowing what type of failure you are dealing with.

This skill diagnoses the failure first. Then it prescribes the right response. Those are two separate steps and they cannot be swapped.

### ### Step 1 — Describe What Went Wrong

The developer describes the problem. The skill listens before doing anything else.

Ask:

```

Describe what is wrong. Be specific:

- What did you expect to happen?
- What happened instead?
- How many times have you tried to fix it already?

```

Read the answer carefully. The number of fix attempts is important — it tells you whether this is a fresh problem or a session that has already gone wrong.

### ### Step 2 — Identify the Failure Mode

Based on the description, determine which of three failure modes this is.

#### #### Failure Mode 1 — A specific thing is broken

**\*\*Signs:\*\***

- The problem is isolated — one component, one function, one route
- The rest of the project works correctly
- This is the first or second attempt at fixing it
- The error message or wrong behaviour is clear and specific

**\*\*What it means:\*\***

This is a normal bug. It has a root cause that can be found and fixed precisely.

**\*\*Response:\*\*** Targeted fix — go to Step 3A.

**####** Failure Mode 2 — The session has gone wrong

**\*\*Signs:\*\***

- Multiple fix attempts have made things worse or created new problems
- The code has become tangled — fixes are patching fixes
- Context in this session is full of failed attempts
- It is no longer clear what the original problem was

**\*\*What it means:\*\***

The session is polluted. More prompting will not help — it will compound the damage. The feature needs to be rebuilt in a clean context, not patched further.

**\*\*Response:\*\*** Hard reset — go to Step 3B.

**####** Failure Mode 3 — The foundation is wrong

**\*\*Signs:\*\***

- The code runs but produces fundamentally wrong behaviour
- Claude has been confidently building something that misunderstands a core requirement, library API, or architectural pattern
- The problem is not a bug in the implementation — the implementation itself is wrong
- Fixing individual pieces will not help because the approach is incorrect

**\*\*What it means:\*\***

This is not a debugging problem. The approach needs to be reconsidered before any code is written. More implementation in the wrong direction makes things harder to untangle.

**\*\*Response:\*\*** Rethink — go to Step 3C.

Tell the developer which failure mode this is before proceeding:

...

This looks like Failure Mode [1/2/3] — [name].

[One sentence explaining why you identified it this way.]

Here is how we handle this:

...

**###** Step 3A — Targeted Fix

For Failure Mode 1.

**####** Diagnose before touching code

Ask the developer to share:

- The exact error message or wrong behaviour
- The specific file or function where it happens
- What the code is supposed to do versus what it actually does

Read the relevant code. Do not read the entire codebase — only what is directly relevant to the problem.

#### Find the root cause

Identify the root cause before suggesting any fix. A root cause is the actual reason the problem exists — not a symptom of it.

State the root cause clearly:

...

Root cause: [specific explanation of why this is happening]

This is different from the symptom because: [explanation]

...

#### Suggest a precise fix

Describe the fix that addresses the root cause. Not a workaround. Not a patch on top of broken code.

...

Fix: [what needs to change and why]

This will resolve the root cause because: [explanation]

...

Wait for the developer to confirm before making any changes.

#### If the fix does not work

If the suggested fix does not resolve the problem — stop. Do not suggest another fix immediately.

Re-examine the root cause diagnosis. If the fix did not work, the root cause was probably wrong. Diagnose again from the beginning before trying again.

If two root cause diagnoses have both been wrong — this may actually be Failure Mode 2 or 3. Re-evaluate.

### Step 3B — Hard Reset

For Failure Mode 2.

#### Acknowledge the situation honestly

...

This session has gone too far in the wrong direction to recover by patching. The right move is a clean start.

This is not a failure — it is the correct response to a polluted context. A fresh session with clear intent will be faster than continuing here.

...

#### Save what is worth keeping

Before the session ends, extract anything valuable from the current state:

- What was the original feature supposed to do?
- What parts of the current implementation, if any, are actually correct?
- What has been learned about what does not work?
- What should the next session avoid?

Write this as a brief reset note:

...

## Reset Note — [Feature Name]

### What we were building  
[Original feature description]

### What went wrong  
[Honest summary of how the session went off track]

### What to avoid next time  
[Specific approaches or patterns that did not work]

### Starting point for next session  
[Where to begin fresh — what to keep, what to discard]

...

#### Instruct the developer

...

Next steps:

1. Save this reset note somewhere accessible
2. End this session completely
3. Start a fresh session
4. Begin with /remember restore if memory exists

5. Approach [feature name] again with the reset note as context

Do not continue in this session.

...

### Step 3C — Rethink

For Failure Mode 3.

#### Name the wrong assumption

The foundation being wrong means something was assumed that should not have been. Find it.

...

The core issue is not a bug — it is a wrong assumption:

Assumed: [what was assumed]

Reality: [what is actually true]

This means the current implementation cannot be fixed by patching. The approach needs to change.

...

#### Propose the correct approach

Based on the correct understanding, describe what the approach should have been:

...

Correct approach: [description]

Key difference from current approach: [explanation]

What needs to be discarded: [what cannot be salvaged]

What can be kept: [what is still valid]

...

#### Do not start rebuilding immediately

A rethink needs the developer to understand and agree before any code changes. Present the analysis and wait for confirmation.

...

Does this diagnosis match your understanding?

If yes — we can start fresh with the correct approach.

If no — tell me what I am getting wrong.

...

Only after the developer confirms does any rebuilding begin.

### ### The Principle

The worst thing you can do when something is broken is keep doing the same thing faster.

Diagnose first. Respond correctly. Different failures need different responses — and knowing which failure you are dealing with is more than half the solution.

---

### ## Remember Skill

---

name: remember

description: Save what matters at the end of a session so the next session picks up exactly where you left off. Or restore context at the start of a new session so nothing is lost between them.

---

AI has no memory between sessions. Every new session starts blank. This skill fixes that.

Run it at the end of a session to save. Run it at the start of a new session to restore. That is all it does — but done consistently, it means nothing ever gets lost.

### ### How to Invoke

**\*\*To save at end of session:\*\***

...

`/remember save`

...

**\*\*To restore at start of new session:\*\***

...

`/remember restore`

...

If the developer just runs `/remember`` without specifying — ask them which one they need.

### ### Save Mode

When the developer runs `/remember save``:

#### #### What to capture

Read the entire session. Extract only what a developer would genuinely need to continue this work in a completely fresh context. Not a transcript. Not a summary of everything that happened. The essential state.

Think like someone handing off a project to a colleague who is equally skilled but knows nothing about what happened today. What would they need to know to continue without losing anything?

Capture:

**\*\*What was built\*\*** — specific files created or modified, features completed, components added. Be precise. Not "built the auth flow" — "created app/(auth)/login/page.tsx, app/(auth)/callback/page.tsx, and middleware.ts. OAuth with Google and GitHub working end to end."

**\*\*Decisions made\*\*** — choices that would be hard to reverse or that future work depends on. Not implementation details — architectural choices. "Chose to use server-side data fetching over client-side — avoids loading states and keeps sensitive logic off the client."

**\*\*Problems solved\*\*** — any issue that took time to figure out. So the next session does not solve the same problem twice. "Third party auth callback requires a trailing slash in the redirect URL — fixed in the callback handler."

**\*\*Current state\*\*** — exactly where things stand right now. What works, what is partial, what is known to be broken.

**\*\*What comes next\*\*** — the very next thing that needs to happen. Specific enough that the next session can start immediately without figuring out where to begin.

**\*\*Open questions\*\*** — anything unresolved that the next session needs to address.

#### What not to capture

- Implementation details that are visible in the code
- Decisions already documented in context files
- Anything that can be inferred by reading the codebase
- The process of how something was built — only what was built and what was decided

#### Where to save

Write the memory to `memory.md` in the project root. Overwrite it completely — do not append. This file always contains only the most recent session state.

#### Format

```markdown

Memory — [Feature or Session Name]

Last updated: [date and time]

What was built

[Specific files, components, features completed this session]

Decisions made

[Architectural and implementation decisions that future work depends on]

Problems solved

[Issues resolved this session — so they are not solved again]

Current state

[Exactly where things stand — what works, what is partial, what is broken]

Next session starts with

[The very first thing to do in the next session — specific and actionable]

Open questions

[Anything unresolved that needs addressing]

...

After writing the file, confirm to the developer:

...

Memory saved to memory.md.

Next session: run `/remember restore` to pick up from here.

...

Restore Mode

When the developer runs `/remember restore` at the start of a new session:

Step 1 — Find the memory

Look for `memory.md` in the project root. If it does not exist, tell the developer:

...

No memory.md found in this project.

Either this is the first session, or the file was not saved.

To save memory at the end of a session, run `/remember save`.

...

Step 2 — Read everything available

Read `memory.md` first. Then read any other context files that exist — CLAUDE.md, context files, whatever is available. Build the most complete picture possible of where this project is and where it is going.

Step 3 — Confirm what was restored

Do not start building. Do not assume the developer wants to continue immediately. Summarise what was restored so the developer can verify Claude understood correctly.

...

Memory restored. Here is where we are:

****Last session:**** [what was built]
****Current state:**** [what works right now]
****Decisions in place:**** [key decisions that are locked]
****Next up:**** [what the next session should start with]

Is this correct? Say yes to continue, or correct anything that does not look right before we proceed.

...

Only after the developer confirms does the session continue.

If memory is incomplete or unclear

If `memory.md` exists but is missing important context, say so honestly:

...

I found memory.md but some context seems missing — [what is unclear or absent].

Should we continue with what we have, or do you want to fill in the gaps before we start?

...

Do not guess. Do not assume. Surface the gap and let the developer decide.

The Rule

Every session ends with `/remember save`.
Every session starts with `/remember restore`.

That is the whole system. Consistent use is what makes it work.
A skill used sometimes is a skill that cannot be relied on.

Review Skill

name: review

description: After building a feature, verify it matches what was planned, respects the system architecture and design standards, and is ready for production. Reports issues clearly so the developer decides what to fix.

Building is not done when the code runs. It is done when the code is correct.

AI moves fast. Fast means things get built that work on the surface but drift from the architecture, violate the design system, or miss edge cases that matter. This skill catches those things before they compound into bigger problems.

Run this after every feature. Before you move on.

What This Skill Does Not Do

It does not fix anything. It reports what it finds and lets the developer decide what matters and what to do about it. Fixing without understanding is how problems get buried, not solved.

Step 1 — Understand What Should Have Been Built

Before reviewing anything, establish the benchmark.

Read in this order:

- The implementation plan from `/architect` if one exists
- The feature description or task that was given
- Any relevant context files — architecture boundaries, code standards, design rules

If no plan exists, ask the developer to describe what the feature was supposed to do before reviewing. You cannot verify correctness without knowing what correct looks like.

Step 2 — Review in Three Layers

Layer 1 — Does it match the plan?

Compare what was built against what was planned.

Check:

- Every part of the feature description — is it all there?
- The decisions made during planning — are they reflected in the code?
- The scope — did the implementation stay within bounds or add things that were not asked for?

Flag anything that was planned but missing. Flag anything that was built but not planned.

Layer 2 — Does it respect the system?

This is where AI drift most commonly happens. The feature works, but it violates rules that the project depends on.

Check:

- **Architecture boundaries** — does code in the right place own the right responsibilities? No UI logic in API routes. No DB calls in components. Whatever the project's boundaries are — are they respected?
- **Design system** — are the correct tokens, classes, and patterns used? Any hardcoded values that should be variables? Any raw color classes that should use the design system?
- **Code standards** — naming conventions, file organisation, TypeScript strictness, error handling patterns — do they match what the project established?
- **Existing patterns** — does this feature introduce a new pattern when an existing one should have been used?

Layer 3 — Is it production ready?

Check:

- Error handling — what happens when things go wrong? Are errors caught and handled or does the feature silently fail?
- Edge cases — empty states, loading states, missing data — are these handled?
- Console errors — any errors or warnings in the browser or terminal?
- Obvious bugs — anything that would clearly break for a real user?

Step 3 — Report What You Found

After completing all three layers, produce a clear report. Do not bury issues. Do not soften them. Report honestly so the developer can make informed decisions.

...

Review — [Feature Name]

Layer 1 — Plan alignment

[PASS / ISSUES FOUND]

[List any gaps between what was planned and what was built]

Layer 2 — System integrity

[PASS / ISSUES FOUND]

[List any architecture, design, or code standard violations]

Layer 3 — Production readiness

[PASS / ISSUES FOUND]

[List any error handling gaps, edge cases, or obvious bugs]

Summary

[X] issues found across [Y] layers.

[If no issues: "No issues found. This feature is ready to ship."]

[If issues: "Resolve the above before moving to the next feature."]

``

Step 4 — Let the Developer Decide

After presenting the report, stop. Do not start fixing. Do not suggest fixes unless the developer asks.

Wait for the developer to:

- Ask you to fix a specific issue
- Tell you an issue is intentional and can be ignored
- Confirm everything is resolved and ready to move on

The developer owns the quality decision. You inform it.

Severity Guide

Not all issues are equal. Use this to help the developer prioritise:

****Critical — fix before moving on****

- Architecture boundary violations that will break future features
- Missing error handling that causes silent failures
- Functionality that was planned but completely missing

****Important — fix soon****

- Design system drift that will cause UI inconsistency
- Code standard violations that will compound across the codebase
- Edge cases that a real user will encounter

****Minor — fix when convenient****

- Naming inconsistencies that do not affect behaviour
- Missing optimisations
- Style issues that do not affect the design system

Label each issue with its severity so the developer can triage quickly.

The Standard

The question this skill answers is not "does it work?"

The question is "is it correct?"

Working and correct are not the same thing. A feature can work today and break the project tomorrow.

Review exists to catch the difference.